1. **A user opens your portfolio from a link. Describe step by step what happens between the browser and the server, and draw a simple client–server diagram (hand-drawn and photographed is fine). Say where the app logic and the database sit.**

Ans: When a user clicks on my portfolio link, the following steps happen between the browser and the server.

Step 1: The user clicks on my portfolio link in a web browser like Chrome.

Step 2: The browser sends a request to the server to open my portfolio website.

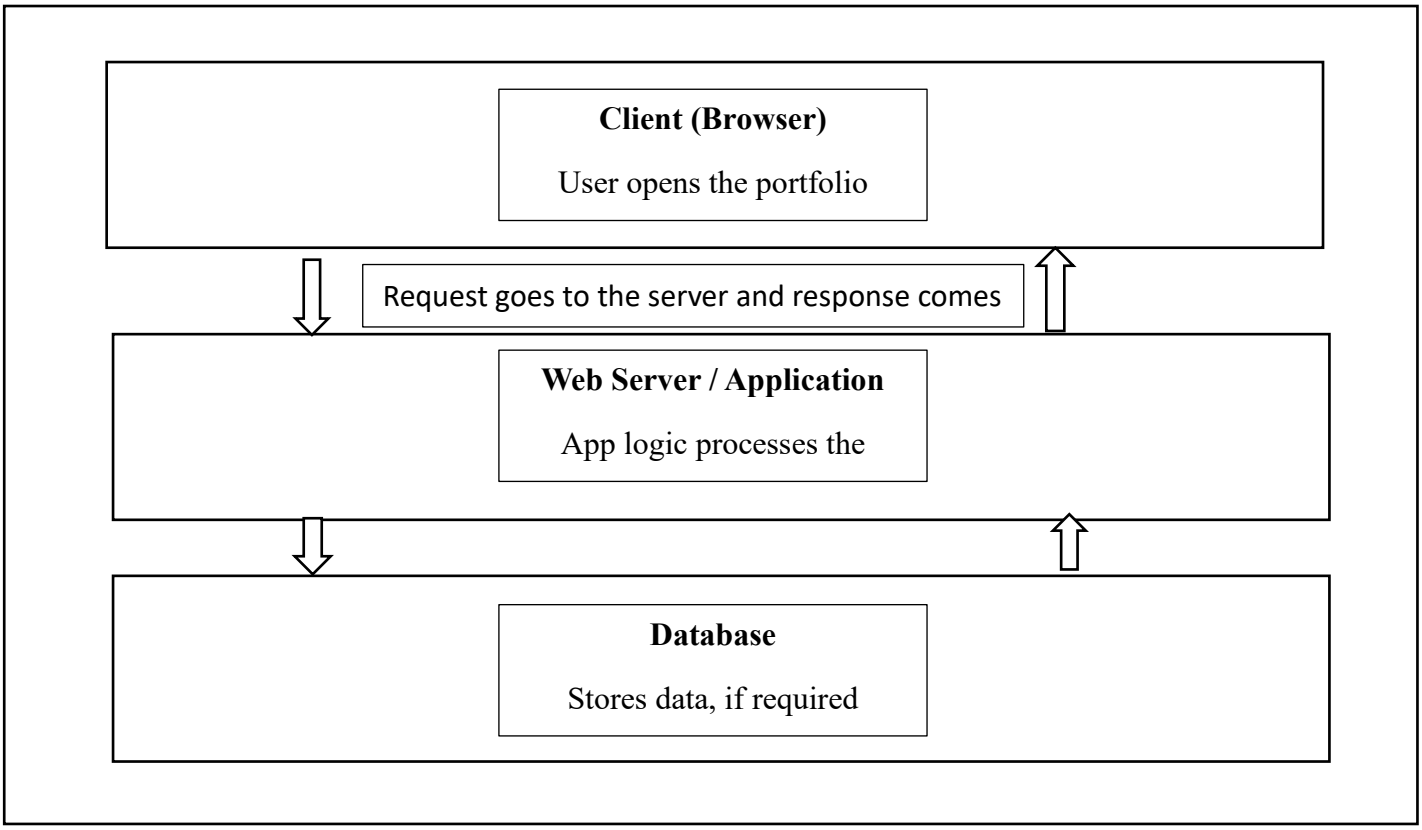
Step 3: The server receives the request and processes it.

Step 4: The server sends the required files, such as HTML, CSS, JavaScript, and images, to the browser.

Step 5: The browser uses these files to display my portfolio website on the screen.

Step 6: If my portfolio needs data from a database, the application logic on the server communicates with the database, fetches the data, and sends the result back to the browser.

Client–Server Diagram:



Where do the app logic and database sit?

- Client (Browser): Displays the portfolio to the user.
- Application logic: Runs on the server and handles requests and processing.
- Database: Stores information such as user details, project information, or contact messages.

2. Your contact form sends a visitor's message to a server. Should it use GET or POST, and why? What happens if method is left out? Name the status code the server returns on success and the family of codes that means the server failed.

Ans: I would use the POST method for my contact form because the visitor is sending a message to the server.

1. **Why POST?** It sends the form data in the request body instead of putting it in the URL. This is better for sending messages and personal details. We should also use HTTPS to protect the data.
2. **What happens if the method is left out?** In an HTML form, the default method is GET. The form data is added to the URL.
3. **Success status code:** 200 OK means the request was successful. If the server creates a new message, it may return 201 Created.
4. **Server failure status codes:** The 5xx family indicates a server-side error. For example, 500 Internal Server Error.

3. Name the three types of CSS and say where each is written. If all three colour the same paragraph differently, which wins? Which type would you use for a style shared by four pages, and why?

Ans: There are three types of CSS used to style a web page.

1. **Inline CSS**: It is written directly inside the HTML element using the style attribute.

Example:

Hello World

2. **Internal CSS:** It is written inside the <style> tag in the <head> section of an HTML page.

Example:

<style>

p {

color: blue;

}

</style>

3. **External CSS:** It is written in a separate .css file and connected to HTML using the <link> tag.

Example:

<link rel="stylesheet" href="style.css">

Which CSS wins if all three style the same paragraph differently?

Generally, inline CSS wins over internal and external CSS when the rules have the same importance and no other priority differences.

Which CSS would I use for four pages?

I would use External CSS because one CSS file can be linked to all four pages. This saves time, avoids repeating code, and makes it easy to maintain a consistent design across all pages.

4. **A page has Hello . Rules exist for *, p, each with a different .note and #intro, color. Which applies, and what is the full order of precedence from strongest to weakest?**

Ans: Different CSS rules are applied to this paragraph.

- * — Universal selector, applies to all elements.
- p — Element selector, applies to all paragraph elements.
- .note — Class selector, applies to elements with the class note.
- #intro — ID selector, applies to the element with the ID intro.

Which color will apply?

The color defined by #intro will apply because an ID selector has higher specificity than a class selector, an element selector, and the universal selector, assuming equal importance and no other priority differences.

Order of precedence (strongest to weakest)

1. !important declaration
2. Inline CSS
3. ID selector (#intro)
4. Class selector (.note)

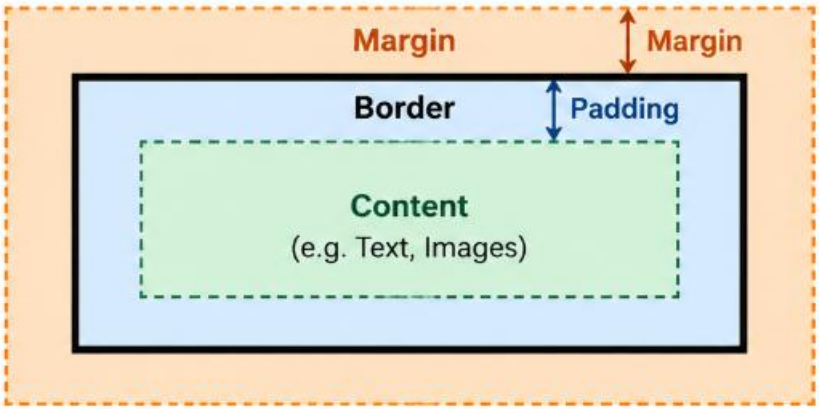
5. Element selector (p)
6. Universal selector (*)

5. **Explain padding and margin with a small drawing of a showing content, padding, border and margin. What does box-sizing: border-box change?**

Ans: 1. Padding: Padding is the space between the content and the border. It creates space inside the element.

2. Margin: Margin is the space outside the border. It creates space between one element and another.

Simple diagram of the CSS Box Model:



What does box-sizing: border-box do?

Normally, the width of an element includes only its content. Padding and border are added to the total width.

When we use box-sizing: border-box, the specified width includes the content, padding, and border. This makes it easier to control the size of elements.

Example:

```
div {
   width: 200px;
   padding: 20px;
   border: 5px solid black;
   box-sizing: border-box;
}
```

6. **Where does JavaScript code run, and what is the JS engine in a browser? Why is no compiler needed to run JavaScript? Name two things JavaScript can do on a web page that HTML and CSS alone cannot.**

Ans:

1. Where does JavaScript code run?

JavaScript code runs in the browser on the user's computer or device. It can also run on a server using environments like Node.js.

2. What is a JavaScript engine?

A JavaScript engine is a program inside the browser that reads and executes JavaScript code. For example, Chrome uses the V8 engine.

3. Why is no separate compiler needed?

JavaScript does not need a separate compiler to run because the browser has a built-in JavaScript engine. The engine interprets and compiles the code internally and executes it.

4. Name two things JavaScript can do that HTML and CSS alone cannot.

1. Form validation: It can check whether the user has entered valid details before submitting a form.
2. Dynamic content updates: It can change text or images when a user clicks a button without reloading the whole page.

7. **JavaScript has no type safety. Explain what happens when you write then a = "hello";. Name the three data types you have learned so far.**

Ans: JavaScript is a dynamically typed language. This means we do not need to declare the data type of a variable separately.

For example:

let a = "hello";

a = 5;

Here, first a stores the string "hello". Later, it stores the number 5. JavaScript allows this because the variable's type can change during execution.

Three data types:

1. String: Stores text, e.g. "hello".
2. Number: Stores numbers, e.g. 5 or 10.5.
3. Boolean: Stores true or false.

8. **Explain the four values of the CSS position property (static, a = 5; and relative, absolute, fixed). When would you use top, left, right and bottom?**

Ans: The CSS position property controls how an element is placed on a web page.

1. Static: This is the default position. The element appears in the normal page layout. The top, left, right and bottom properties do not move it.

2. Relative: The element stays in its normal layout position but can be moved from its original position using top, left, right or bottom.

3. Absolute: The element is removed from the normal layout and positioned relative to its nearest positioned ancestor. If there is no such ancestor, it is generally positioned relative to the initial containing block.

4. Fixed: The element is positioned relative to the browser viewport. It usually stays in the same place even when the page is scrolled.

When do we use top, left, right and bottom?

These properties specify the distance from the corresponding edge of the element's positioning reference.

Example:

```
.box {
   position: fixed;
   top: 10px;
   right: 20px;
}
```

9. **A container has display: flex and three boxes with flex values of 1, 2 and 1. How is the width shared between the boxes? What do gap and justify-content: center do?**

Ans:

1. How is the width shared between the three boxes?

When a container has display: flex and the three boxes have flex values of 1, 2, and 1, the available space is divided in the ratio 1 : 2 : 1.

- Box 1 gets 25% of the available space.
- Box 2 gets 50% of the available space.
- Box 3 gets 25% of the available space.

This assumes the boxes have no other size constraints and that we are dividing the available space after accounting for any gap.

Example:

```
.container {
   display: flex;
}


.box1 { flex: 1; }
.box2 { flex: 2; }
.box3 { flex: 1; }
```

2. What does gap do?

The gap property adds space between the boxes.

Example: gap: 20px; adds 20 pixels of space between adjacent boxes.

3. What does justify-content: center do?

It places the group of boxes in the center along the main axis. In a normal row-based flex container, this means horizontal centering.